

## R4 — O display, e o custo da espera

*Um protocolo gerado por software, medido no pino*

Microcontroladores — DENE/UFMT

### OBJETIVOS

- Justificar cada atraso do driver a partir dos mínimos da folha de dados, e não por cópia.
- Medir no osciloscópio o pulso de habilitação e o tempo de uma atualização de tela.
- Provocar, observar e explicar a falha intermitente causada por inicialização apressada.
- Comparar quatro bits contra oito bits **na mesma bancada**, e decidir com o número.

|                        |                                         |           |
|------------------------|-----------------------------------------|-----------|
| CH2-1 (LCD)            | ON — é o único roteiro em que ele entra | NESTE KIT |
| CH5-3 e CH5-4          | OFF, obrigatoriamente                   |           |
| Chaves SWITCHS (PORTB) | Todas em OFF                            |           |

### 1 O primeiro periférico que não obedece na hora

Até o R3, escrever num pino produzia efeito imediato. O display não: ele tem controlador próprio, oscilador próprio, e leva tempo para executar o que recebe.

| Operação                   | Tempo mínimo | Em ciclos de máquina |
|----------------------------|--------------|----------------------|
| Comando comum ou caractere | ~ 40 $\mu$ s | 160                  |
| Limpar o display           | ~ 1,6 ms     | 6 400                |
| Energização inicial        | ~ 40 ms      | 160 000              |

A última linha é o mínimo da folha de dados. O driver espera 50 ms — a diferença é margem, e a Tarefa 5 mostra por que ela não é opcional.

Ignorar esses tempos produz o sintoma mais traiçoeiro do semestre: **o display funciona às vezes**.

ATENÇÃO

Um sistema que funciona quase sempre é mais perigoso que um que nunca funciona. O defeito não aparece na bancada — aparece em campo, sob condições que ninguém consegue reproduzir.

E há um efeito colateral visível: no XM118 os dados do display usam **o mesmo PORTD dos oito LEDs**. Escrever no display faz os LEDs piscarem.

NOTA

Isso é comportamento esperado, não defeito, e ilustra o que a §3 do R1 já dizia: pinos são recurso escasso, e o compartilhamento tem consequências visíveis.

### 2 Parte 1 — o driver, lido antes de usado

O professor fornece `lcd.c` e `lcd.h`. **Leia antes de gravar qualquer coisa.**

Os comentários do fonte marcam com [T#] a tarefa que trata de cada trecho. Eles fazem as perguntas; nenhum as responde.

**Tarefa 1.** Localize `lcd_pulso()` e responda:

TAREFA

(a) Por que existem dois atrasos, um com o pino E alto e outro depois de baixá-lo?

(b) Cada um deles atende a qual mínimo da folha de dados do HD44780? Nomeie os dois.

(c) A folha de dados pede 230 ns de largura de pulso. `__delay_us(1)` produz 1000 ns. Por que não usar um valor menor, já que um único ciclo de máquina — 250 ns — já bastaria?

A resposta de (c) é a que separa copiar de projetar.

CONCEITO

Um ciclo de máquina atenderia quatro dos cinco mínimos. O `__delay_us(1)` é escolhido por **margem**: dá folga de mais de quatro vezes sobre o pior caso, o que absorve variação de módulo, de temperatura e de tensão sem que ninguém precise medir nada.

E há uma razão concreta: módulos alimentados a 2,7 V, e boa parte dos clones vendidos hoje, especificam 450 ns em vez de 230. Ainda cabe em 1000 ns; não caberia num pulso de 250 ns.

**O ganho de “otimizar” o pulso seria de 750 ns por nibble. O preço seria um driver que funciona na sua bancada e falha na do colega, com um módulo de outro lote.**

**Tarefa 2.** Duas ponteiros, e é este roteiro que exige as duas.

TAREFA

| Canal | Ponto         | O que ele mostra                                         |
|-------|---------------|----------------------------------------------------------|
| CH1   | E — pino RE1  | O pulso de habilitação. Dispare aqui, na borda de subida |
| CH2   | D6 — pino RD6 | Um bit do barramento de dados                            |

Escreva o caractere 'A' continuamente, e nada mais. 'A' é 0x41, ou 0100 0001 em binário: o nibble alto tem D6 em **1** e o nibble baixo tem D6 em **0**. Os dois pulsos de um mesmo byte ficam, portanto, visivelmente diferentes no CH2 — e é isso que permite dizer qual veio primeiro.

Meça, e preencha:

| Grandeza                                 | Previsto (P4) | Medido |
|------------------------------------------|---------------|--------|
| Largura do pulso de habilitação          |               |        |
| Intervalo entre dois pulsos consecutivos |               |        |
| Tempo entre os dois nibbles de um byte   |               |        |
| D6 estável <b>antes</b> da descida de E  |               |        |
| D6 mantido <b>depois</b> da descida de E |               |        |

Compare as três primeiras com a previsão P4 da folha da aula 3, e as duas últimas com  $t_{\text{sub}}[\text{DSW}]$  e  $t_{\text{sub}}[\text{H}]$  da folha de referência.

**(a)** Qual nibble sai primeiro, o alto ou o baixo? Responda **pela medida**, e diga o que na tela do osciloscópio permitiu decidir.

**(b)** O dado muda antes ou depois da borda de descida de E? Por que a resposta tinha de ser essa?

Com uma ponteira só, “o dado é capturado na borda de descida” é uma afirmação que o aluno aceita. Com duas, ela é uma coisa que ele vê: o CH2 já está parado e estável quando o CH1 desce, e só muda bem depois.

CONCEITO

É a diferença entre saber o protocolo e ter medido o protocolo — e é também como se depura um barramento que não funciona, quando o problema é justamente o dado chegar tarde demais.

As duas ponteiros compartilham o mesmo terra pelo chassi do osciloscópio. Prenda as duas garras no **mesmo** ponto de terra do kit. Terras em pontos diferentes da placa fecham uma malha, e a malha aparece como ruído exatamente na escala de tempo que você está tentando medir.

ATENÇÃO

### 3 Parte 2 — a inicialização, e a falha intermitente

**Tarefa 3.** Localize a sequência de inicialização em `lcd_iniciar()` e descreva o que cada comando faz:

TAREFA

|                         |       |
|-------------------------|-------|
| <b>0x30, três vezes</b> | _____ |
| 0x20                    | _____ |
| 0x28                    | _____ |
| 0x0C                    | _____ |
| 0x06                    | _____ |
| 0x01                    | _____ |

Use a folha de dados do HD44780 para justificar cada resposta. A resposta não é o que o código diz: é o que o comando **faz** no controlador.

**Tarefa 4.** Troque 0x0C por 0x0F e observe. O que mudou, e por quê?

TAREFA

**Tarefa 5.** Remova o `__delay_ms(LCD_T_ENERGIA_MS)` de `lcd_iniciar()` — são os 50 ms — e grave.

TAREFA

O display funciona? Funciona **sempre**? Ligue e desligue o kit três vezes:

|                    |       |
|--------------------|-------|
| <b>Tentativa 1</b> | _____ |
| Tentativa 2        | _____ |
| Tentativa 3        | _____ |

Este teste é o ponto central do roteiro, e o resultado costuma ser: funciona duas vezes e falha uma.

CONCEITO

Os 50 ms não são para o display: são para a **alimentação**. O controlador só garante o estado interno depois que a tensão estabiliza, e quem liga o kit pela USB não controla quanto tempo isso leva. Em alguns arranques a tensão sobe rápido o bastante; em outros, não.

**Você não pode testar a ausência dessa falha.** Só pode garantir a margem.

#### 3.1 Os três 0x30

**Tarefa 6.** Remova **uma** das três repetições de 0x30. Grave, e ligue e desligue o kit cinco vezes. Depois pressione o reset — sem desligar — mais cinco vezes.

TAREFA

Os dois casos se comportam igual? Confronte com a previsão P3 da folha da aula 3.

Não se comportam, e a diferença é o conteúdo da aula 3.

CONCEITO

Ao **energizar**, o controlador está sempre no mesmo estado: acabou de nascer, em oito bits. Duas repetições bastam, e a versão errada parece correta.

Ao **pressionar o reset**, o microcontrolador reinicia e o display não — ele continua em quatro bits, e pode estar esperando o nibble alto ou o nibble baixo. São três estados possíveis, e o terceiro só converge no terceiro pulso.

**Testar apenas na energização esconde exatamente o defeito que a terceira repetição existe para evitar.** É o mesmo padrão da Tarefa 5, com outro mecanismo.

### 4 Parte 3 — números no display

O display recebe **caracteres**, não números. Para mostrar 37 é preciso enviar '3' e depois '7'.

#### 4.1 O número que você tem não tem dígitos

Um `uint8_t` é um byte. Ele guarda uma quantidade entre 0 e 255, e guarda essa quantidade em binário — oito bits, peso 128 no mais significativo. O valor 37 é 0010 0101, e não há nenhum 3 nem nenhum 7 armazenado em lugar nenhum.

Os dígitos decimais não estão no número: eles são uma **representação** dele. E a representação decimal é uma soma de potências de dez:

$$n = c \cdot 10^2 + d \cdot 10^1 + u \cdot 10^0$$

com  $c$ ,  $d$  e  $u$  entre 0 e 9. Exibir um número é resolver essa equação para  $c$ ,  $d$  e  $u$  — é o mesmo trabalho que o encontro 2 fez em sentido contrário, quando converteu hexadecimal em binário.

Repare que '3' não é 3. O caractere '3' vale 0x33 na tabela do HD44780, e o que faz '0' + dígito funcionar é os dez dígitos serem consecutivos a partir de 0x30. Encontrar o dígito e converter o dígito em caractere são dois problemas diferentes, e só o segundo é barato.

NOTA

#### 4.2 Três maneiras de achar os dígitos

**A clássica: divisão e resto.** Cada divisão por dez descasca um dígito pela direita, e o resto é o dígito.

```
u = n % 10;  n = n / 10;
d = n % 10;  n = n / 10;
c = n;
```

Lê-se bem, e é o que quase todo mundo escreve. Mas o PIC18 **não tem instrução de divisão**. Ele tem multiplicador em hardware — `MULWF` faz 8 por 8 em um ciclo — e não tem divisor. Cada `/` e cada `%` deste trecho é uma chamada a uma rotina da biblioteca, que divide por subtrações sucessivas num laço.

**A ingênua: subtrair até não dar mais.** É o que a rotina da biblioteca faz, só que escrito à mão:

```
c = 0; while (n >= 100u) { n -= 100u; c++; }
d = 0; while (n >= 10u)  { n -= 10u;  d++; }
u = n;
```

Duas coisas valem observar. A primeira é que isto é a divisão, e não uma alternativa a ela. A segunda é que o número de voltas **depende do valor**: para  $n = 9$  o segundo laço não executa nenhuma vez, e para  $n = 99$  ele executa nove. O custo não é constante — e um trecho de código cujo tempo depende do dado é exatamente o que estraga uma temporização.

**A elegante: multiplicar pela recíproca.** Dividir por dez é multiplicar por um décimo. Como não há ponto flutuante, usa-se uma fração cujo denominador é potência de dois, porque dividir por potência de dois é deslocar bits:

$$\frac{1}{100} \approx \frac{41}{4096} \quad \frac{1}{10} \approx \frac{205}{2048}$$

```
c = (uint8_t)((n * 41u) >> 12); /* n / 100 */
n = (uint8_t)(n - c * 100u);
d = (uint8_t)((n * 205u) >> 11); /* n / 10  */
u = (uint8_t)(n - d * 10u);
```

Sem laço, sem divisão, e sempre o mesmo número de ciclos.

**Por que 41 e 205, e por que isso dá o resultado exato.**

CONCEITO

41/4096 vale 0,010009..., e não 0,01. A multiplicação erra para cima — e é por isso que ela funciona: o deslocamento à direita trunca, e o erro é pequeno demais para empurrar o resultado para o degrau seguinte. Dentro da faixa em que isso vale, o resultado não é aproximado: é **idêntico** ao da divisão inteira.

E a faixa importa. Para `uint8_t`, de 0 a 255, as duas constantes acima são exatas. A de 41 continua exata até 1023 — e falha pela primeira vez em 1099. Um número mágico carrega sempre, junto, a faixa em que ele é válido, e usar fora dela produz um erro silencioso.

**O produto intermediário é o perigo, e ele é o mesmo defeito do encontro 4.**

ATENÇÃO

$n * 205$  com  $n = 255$  dá 52275, que ainda cabe em dezesseis bits. Mas se alguém aplicar a mesma linha a um valor de dez bits — 1023, por exemplo — o produto vale 209715 e **não cabe**. A conta dá voltas em silêncio e devolve um dígito plausível e errado.

No encontro 5 vocês vão reencontrar isso literalmente, ao multiplicar o código do conversor por 625. É a mesma armadilha, com outro número: **toda multiplicação por constante grande precisa de uma verificação do maior produto possível.**

**Tarefa 7.** Escreva a sua própria `lcd_u8(uint8_t v)`, que mostre um valor de 0 a 255, e use para exibir um contador incrementando a cada 500 ms.

TAREFA

Escreva a versão ingênua: só os dígitos que o número tem, sem nenhum cuidado com o que já estava na tela. O driver traz uma `lcd_numero()` pronta — **não use ainda**, e não a chame de dentro da sua.

(a) Implemente a separação de dígitos das **três** maneiras da seção anterior: divisão e resto, subtrações sucessivas, e multiplicação pela recíproca. Confira que as três dão o mesmo resultado para 0, 7, 99, 100 e 255.

(b) Meça as três, com o pino auxiliar levantado antes e baixado depois da separação — **sem** a escrita no display, que domina tudo:

| Implementação            | $v = 7$ | $v = 255$ |
|--------------------------|---------|-----------|
| Divisão e resto          |         |           |
| Subtrações sucessivas    |         |           |
| Recíproca e deslocamento |         |           |

Duas colunas, e é de propósito: **qual das três muda de tempo conforme o valor, e por quê?**

(c) Antes de escrever qualquer coisa: por que `'0' + dígito` funciona? O que garante que os dígitos sejam consecutivos na tabela de caracteres?

Compile com otimização e confira o resultado no listing de assembly antes de concluir qualquer coisa da medida. O XC8 reconhece multiplicação por constante e costuma trocá-la por somas e deslocamentos sozinho — o que significa que a terceira versão pode não ser tão diferente da primeira quanto o código sugere, e que a primeira **continua** chamando a rotina de divisão.

NOTA

O ponto da tarefa não é que a recíproca é sempre melhor. É que você não sabe qual é melhor até medir, e que o compilador é um participante da conta.

**Tarefa 8.** Deixe o contador passar de 100 para 99. O que aparece?

TAREFA

Compare com a previsão P1 da folha e explique. Depois corrija — **de duas formas diferentes**, uma na função de escrita e outra na formatação do número.

Só então abra a `lcd_numero()` do driver e diga qual das duas correções ela usa, e por que é ela que o resto do semestre vai chamar.

Este é o mesmo defeito da P1, com outro número: a segunda cadeia é mais curta que a primeira, e o display não apaga nada. Sobra o excedente do texto anterior.

NOTA

A correção que o resto do semestre vai usar é largura fixa — o número ocupa sempre as mesmas posições, completado com espaço à esquerda.

## 5 Parte 4 — quatro bits contra oito, medido

O kit liga as oito linhas de dados ao PORTD, então os dois modos são possíveis nesta bancada. A aula 3 argumentou que quatro bits economizam quatro pinos. Agora vocês medem o que isso custa.

**Tarefa 9.** Rode o mesmo texto nos dois modos e meça, com um pino auxiliar levantado antes e baixado depois da escrita:

TAREFA

|                          | Oito bits | Quatro bits |
|--------------------------|-----------|-------------|
| Pulsos de E por byte     | _____     | _____       |
| Tempo de um caractere    | _____     | _____       |
| Tempo de uma linha de 16 | _____     | _____       |
| Pinos ocupados           | _____     | _____       |

O resultado costuma surpreender: **quatro bits quase não custam tempo.**

CONCEITO

Cada byte gasta um pulso a mais, o que acrescenta poucos microssegundos — contra os 40  $\mu$ s de execução do comando, que o controlador consome de qualquer jeito. A diferença fica na casa de 5%.

E economiza quatro pinos, que num projeto com sensor, aquecedor, ventoinha, teclado e comunicação são a diferença entre caber e não caber.

É por isso que **quatro bits é o padrão**, e não porque alguém queira escrever mais código. A decisão se justifica com os dois números medidos, e não com preferência.

## 6 Parte 5 — o custo da tela inteira

**Tarefa 10.** Meça o tempo de uma atualização completa: posicionar o cursor, escrever dezesseis caracteres, posicionar de novo, escrever mais dezesseis.

TAREFA

Levante um pino auxiliar antes e baixe depois; meça a largura do pulso.

|                                                 |       |
|-------------------------------------------------|-------|
| <b>Tempo medido de uma atualização completa</b> | _____ |
| Em ciclos de máquina, a 250 ns                  | _____ |
| Percentual do processador a cada 100 ms (P5)    | _____ |
| Percentual do processador a cada 500 ms (P5)    | _____ |

**Tarefa 11.** Acrescente `lcd_comando(0x01)` — limpar — antes de cada atualização, e meça de novo. Depois olhe para a tela e descreva o que mudou visualmente.

TAREFA

Os dois efeitos aparecem juntos: o tempo sobe alguns milhares de ciclos, e a tela **treme**, porque fica em branco por um instante a cada ciclo.

CONCEITO

A prática correta é reposicionar o cursor e sobrescrever, com largura constante — que é a mesma correção da Tarefa 8, agora por um segundo motivo.

Guarde o número da Tarefa 10. Ele é o mais caro que este curso vai encontrar até o encontro 10, e é a unidade de comparação de tudo o que vem depois: uma conversão analógica custa sessenta ciclos, um tratamento de interrupção custa algumas dezenas, e uma atualização de tela custa alguns milhares.

NOTA

## 7 Armadilhas frequentes

| Sintoma                              | Causa provável                                            |
|--------------------------------------|-----------------------------------------------------------|
| Display aceso, sem texto             | Contraste, ou CH2-1 em OFF                                |
| Linha superior com quadrados escuros | Inicialização incompleta                                  |
| Funciona às vezes                    | Falta de atraso após a energização                        |
| Caracteres embaralhados              | Falta de espera após o comando de limpar                  |
| Texto na linha errada                | Endereço de cursor: a <b>segunda</b> linha começa em 0x40 |
| Resíduo ao diminuir dígitos          | Largura variável; não limpou a posição anterior           |
| LEDs piscando junto                  | Esperado — PORTD é compartilhado                          |
| Relé chaveando a cada caractere      | CH5-3 ou CH5-4 ligado. <b>Desligue agora</b>              |

## 8 Entrega

**Entrega.** As três tabelas de medida (Tarefas 2, 9 e 10), o registro das três tentativas da Tarefa 5, e o resultado da Tarefa 6 nos dois casos — energização e reset.

TAREFA

Responda também:

(a) Por que `__delay_us(1)` e não um único ciclo de máquina? Cite o mínimo que justifica a escolha e a margem obtida.

(b) Na Tarefa 6, por que o defeito aparece no reset e não na energização? Sua previsão P3 acertou a condição de falha?

(c) Com os números da Tarefa 9, você escolheria quatro ou oito bits para um produto? Justifique com as duas grandezas.

(d) A previsão P2 dizia respeito a trocar LATD por PORTD no driver. Você testou? O que aconteceu, e por quê?

**No R5:** a entrada e o conversor. Vocês vão ler um botão que não responde, e descobrir por quê — a resposta está no que existe atrás do pino, e não no código.

NOTA

E o LM35 entra: temperatura na tela, com a conta que vocês trouxeram da última sessão.